

HOSPITAL MANAGEMENT SYSTEM

MeroSwasthya Project Report

A role-based healthcare platform for hospitals, doctors, patients, administrators, appointments, chat, and medical reports.

PROJECT

MeroSwasthya Hospital Management System

REPORT DATE

May 12, 2026

APPLICATION TYPE

MERN-style web application

CURRENT STATUS

Functional development build verified

Prepared as a manager-facing technical and functional summary of the current codebase in `MeroSwasthya-app`.

Table of Contents

1. Executive Summary	9. Database Design
2. Objectives and Scope	10. API Summary
3. Technology Stack	11. Security and Access Control
4. System Architecture	12. Setup and Configuration
5. User Roles	13. Verification Status
6. Functional Modules	14. Limitations and Risks
7. Frontend Implementation	15. Future Scope
8. Backend Implementation	16. Conclusion

1. Executive Summary

MeroSwasthya is a web-based Hospital Management System designed to connect patients, doctors, hospital administrators, and system administrators in a single application. The platform supports patient registration, OTP-based verification, hospital discovery, doctor search, queue-aware appointment booking, doctor approvals, hospital doctor management, medical report creation, visit history, profile image management, and real-time patient-doctor chat.

The application is split into a Node.js/Express backend with MongoDB persistence and a React/Vite frontend. The backend provides REST APIs, JWT-based authentication, role-based authorization, file upload support, OTP email delivery, and Socket.IO-based chat. The frontend provides a clean role-based user experience with a landing page, authentication screens, protected dashboards, responsive layouts, hospital images, doctor and patient avatars, testimonials, and structured report views.

Business Value

Centralizes appointment, hospital, doctor, and patient record workflows into one accessible healthcare platform.

Operational Value

Allows hospitals to manage doctors, availability, and bookings while doctors can handle appointments, reports, and patient communication.

Patient Value

Patients can browse hospitals, book available doctors, chat, and review visit history with medical reports.

Technical Value

Uses a modular backend and role-aware frontend, making the project easier to extend and maintain.

2. Objectives and Scope

Project Objectives

- Provide a digital platform for healthcare appointment and record management.
- Support separate workflows for patients, doctors, hospital users, and admins.
- Allow hospitals to manage doctors and date-specific availability limits.
- Allow patients to search hospitals, select doctors, book appointments, and view medical reports.
- Enable doctors and hospitals to create reports for confirmed or completed appointments.
- Enable real-time patient-doctor messaging after patients initiate a conversation.
- Offer a modern, responsive frontend suitable for demonstration and further development.

In-Scope Functionality

- User registration, login, OTP verification, resend OTP, password reset, and current-user lookup.
- Role-specific dashboards for admin, hospital, doctor, and patient users.
- Doctor profile creation, update, and admin approval.
- Hospital creation, update, doctor search, doctor assignment, and availability management.
- Appointment booking with queue number assignment and date-based limits.
- Medical report creation, report listing, report detail, and patient visit history.
- Profile image upload/delete and static serving of uploaded images.
- Socket.IO chat between patients and doctors.
- Landing page with hospital hero image, sectioned content, testimonials, and CTA.

3. Technology Stack

Layer	Technology	Purpose
Frontend	React 18, React Router, Vite	Single page application, routing, build tooling, responsive UI.
Backend	Node.js, Express 5	REST API server, routing, middleware, static upload serving.
Database	MongoDB, Mongoose	Document persistence, schema definitions, references, indexes.
Authentication	JWT, bcryptjs	Token-based sessions and password hashing.
Email	Nodemailer	OTP verification and password reset email delivery.
Realtime	Socket.IO, socket.io-client	Patient-doctor chat messaging.
Uploads	Multer	Profile image upload with type and size validation.

4. System Architecture

The application follows a client-server architecture. The React frontend communicates with the Express backend through REST endpoints under `/api`. Authentication is handled through JWT tokens stored in frontend local storage. Socket.IO uses the same JWT token during the socket handshake to authenticate realtime chat connections. MongoDB stores user, doctor, hospital, appointment, report, patient profile, and chat records.

Frontend Application

Contains landing page, auth pages, protected app shell, role-based dashboard rendering, API client, chat panel, and shared UI components.

Backend API

Provides REST route groups for auth, admin, doctor, hospital, patient, appointment, report, and chat workflows.

Realtime Chat

Socket.IO server authenticates clients with JWT and emits conversation/message events to user-specific rooms.

MongoDB Models

Mongoose schemas define data relationships between users, doctors, hospitals, appointments, reports, and messages.

High-Level Flow

1. A user registers and verifies the account through email OTP.
2. The user logs in and receives a JWT token.
3. The frontend stores the token and routes the user to the protected dashboard.
4. The dashboard shown depends on the user's role.
5. Protected API calls include the JWT token in the Authorization header.
6. Backend middleware validates the token and checks role permissions.
7. MongoDB stores the resulting data changes.
8. Chat uses Socket.IO events authenticated by the same JWT identity.

5. User Roles

Role	Main Capabilities
Patient	Maintain patient profile, search hospitals, view hospital doctors, book appointments, chat with doctors, view visit history and reports.

Role	Main Capabilities
Doctor	Create/update doctor profile, wait for admin approval, view appointments, update appointment status, chat with patients, create medical reports.
Hospital	Create/update hospital profile, search/link doctors, manage doctor availability, view hospital appointments, create hospital-linked reports.
Admin	View system stats, create users, create hospitals, approve doctors, review reports across the system.

6. Functional Modules

6.1 Authentication and Account Management

- Registration for patient and doctor users.
- Admin registration is protected by an admin secret check.
- Email OTP verification is required before login.
- Login returns JWT token and role-aware redirect metadata.
- Password reset uses email OTP and new password submission.
- Users can upload and delete profile images.

6.2 Admin Module

- System overview with counts for total users, patients, doctors, and hospitals.
- Create doctor, hospital, patient, or admin users.
- Create hospitals and assign hospital admin users.
- Approve pending doctor profiles.
- Review system-wide medical reports.

6.3 Hospital Module

- Hospital users can create and update hospital information.
- Hospitals can search registered doctor-role users.
- Hospitals can link doctors to hospitals and set availability dates.
- Availability supports date-specific patient booking limits.
- Hospitals can view appointments under their hospitals.
- Hospitals can create reports for eligible appointments.

6.4 Doctor Module

- Doctors can create and update profiles with specialty, license, experience, fees, and capacity.
- Doctors become bookable after admin approval.

- Doctors can confirm, complete, cancel, or mark appointments as no-show.
- Doctors can create or update medical reports for confirmed/completed appointments.
- Doctors can reply to patient chats after the patient starts the conversation.

6.5 Patient Module

- Patients can save demographic and medical basics such as blood group, allergies, chronic conditions, and emergency contacts.
- Patients can browse hospital cards with images and contact information.
- Patients can open hospital details, search doctors, choose a doctor, and book appointments.
- Patients can track appointment status and queue/token numbers.
- Patients can view visit history and open reports directly from the visit item.
- Patients can search doctors and start chat conversations.

6.6 Appointment Booking

Appointments use a queue-based approach. The backend calculates the next queue number for a doctor, hospital, and date combination. Active statuses such as pending and confirmed count against the booking limit. The system generates a scheduled time from the queue number using a fixed 30-minute slot model.

6.7 Medical Reports

Medical reports are tied to appointments and can include diagnosis, prescription items, test recommendations, follow-up date, notes, attachments, and creator role. A report is created or updated per appointment, preventing duplicate reports for the same visit.

6.8 Realtime Chat

Chat is available between patients and doctors. Patients initiate the first message. Doctors can reply after a conversation exists. Messages are stored in MongoDB and delivered in realtime through Socket.IO rooms scoped to user IDs.

7. Frontend Implementation

The frontend is a Vite React application. It uses React Router for navigation and a custom authentication context for storing user session data. The API client centralizes base URL handling, token headers, form-data support, and asset URL generation.

Main Pages

Page	Purpose
/ Landing Page	Manager-friendly public entry page with hospital hero image, feature boxes, role cards, testimonials, and CTA.
/login	User login with role-aware app entry after authentication.
/register	Account registration for supported public user roles.
/verify-otp	OTP verification and resend flow.
/app	Protected dashboard shell; content switches by role.

Recent UI Enhancements

- Added a full landing page with hospital image hero, testimonials, and CTA sections.
- Improved global design system: cleaner cards, tabs, forms, notices, and responsive layouts.
- Removed raw JSON API response panels from dashboards.
- Changed patient history to open reports from each visit instead of a separate reports list.
- Reduced hospital search card image sizes for better browsing.
- Added doctor, patient, and testimonial profile images where applicable.
- Improved chat UI with conversation and sender avatars.

8. Backend Implementation

The backend is a CommonJS Express application. The server initializes MongoDB, configures JSON parsing, CORS, upload static serving, REST routes, and Socket.IO. Business logic is organized into route, controller, service, model, middleware, utility, and socket layers.

Backend Folder Structure

Folder/File	Responsibility
server.js	Express app setup, HTTP server, Socket.IO, middleware, and route mounting.
db.js	MongoDB connection setup using Mongoose.
routes/	Endpoint definitions and middleware composition.
controllers/	Request validation, business workflow orchestration, response handling.
models/	Mongoose schemas for application entities.

Folder/File	Responsibility
middleware/	JWT auth, role checks, and upload handling.
services/	Shared domain services, currently chat service logic.
sockets/	Socket.IO chat authentication and event handling.
utils/	Email service and dashboard path helpers.

9. Database Design

Model	Purpose	Key Fields
User	Stores login identity and role.	name, email, password, role, isVerified, OTP fields, phone, profileImage.
PatientProfile	Stores patient-specific health and contact details.	user, dateOfBirth, gender, bloodGroup, address, emergency contact, allergies, chronicConditions.
Doctor	Stores doctor profile data.	user, specialty, licenseNumber, experienceYears, hospitalId, bio, consultationFee, maxDailyBookings, isApproved.
Hospital	Stores hospital details and ownership.	name, address, phone, email, bannerImage, adminUser, isActive.
DoctorHospital	Links doctors to hospitals with availability.	doctor, hospital, maxDailyBookings, availableDates, availabilitySlots, isActive.
Appointment	Stores booked appointments and queue details.	patientUser, doctor, hospitalId, scheduledAt, queueDate, queueNumber, reason, status.
MedicalReport	Stores diagnosis and treatment report tied to an appointment.	appointment, patientUser, doctor, hospitalId, diagnosis, prescription, tests, followUpDate, notes.
ChatMessage	Stores patient-doctor messages.	patientUser, doctor, senderUser, message, timestamps.

10. API Summary

Route Group	Important Endpoints	Purpose
/api/auth	register, login, verify-otp, resend-otp, forgot-password, reset-password, me, profile-image	Authentication, OTP, password reset, current user, and profile image management.

Route Group	Important Endpoints	Purpose
/api/admin	stats, users, hospitals	Admin overview and system management operations.
/api/doctor	doctor listing, create profile, my profile, pending approvals, approve doctor	Doctor directory, doctor self-service, and admin approval.
/api/hospital	hospital listing, mine, doctor-candidates, hospital doctors, create/update hospital, link doctors	Hospital profile, search, doctor assignment, and availability workflows.
/api/patient	me, me/history	Patient profile and visit history with reports.
/api/appointment	book, book-from-hospital, book-by-specialty, my, status update	Appointment creation, queueing, listing, and status updates.
/api/report	create/update, my reports, report detail	Medical report creation and retrieval.
/api/chat	conversations, messages, send message	Chat history and message sending through REST; realtime handled by Socket.IO.

11. Security and Access Control

Current Controls

- Passwords are hashed using bcrypt before storage.
- JWT tokens authenticate protected REST endpoints.
- Role middleware restricts routes to allowed roles.
- Socket.IO validates JWT token before allowing chat events.
- Multer validates profile image MIME type and file size.
- Report access is checked by patient, doctor, hospital, or admin ownership rules.
- Hospital users are restricted to hospitals linked to their account.

Recommended Production Hardening

- Restrict CORS to known frontend origins instead of allowing all origins.
- Use cryptographically secure OTP generation and store hashed OTP values.
- Add rate limiting for login, OTP, password reset, and chat endpoints.
- Move tokens to httpOnly cookies if the deployment model supports it.
- Add centralized error logging and audit trails for sensitive actions.

- Validate request payloads with a schema validation library.

12. Setup and Configuration

Backend Setup

1. Open terminal in `backend`.
2. Install dependencies with `npm install`.
3. Create a `.env` file.
4. Run development server with `npm run dev`.
5. Run production server with `npm start`.

Backend Environment Variables

Variable	Purpose
<code>MONGO_URI</code>	MongoDB connection string.
<code>JWT_SECRET</code>	Secret used to sign and verify JWT tokens.
<code>EMAIL_USER</code>	Email account username for OTP delivery.
<code>EMAIL_PASS</code>	Email account password or app password for OTP delivery.
<code>EMAIL_FROM</code>	Optional sender address.
<code>ADMIN_SECRET</code>	Secret used to allow admin-role registration.
<code>PORT</code>	Optional backend port, default is 5000.

Frontend Setup

1. Open terminal in `frontend`.
2. Install dependencies with `npm install`.
3. Run development server with `npm run dev`.
4. Create production build with `npm run build`.

Frontend Environment Variables

The frontend API base URL defaults to `http://localhost:5000/api`. It can be overridden using `VITE_API_BASE_URL`.

13. Verification Status

The current frontend build was verified successfully with:

`npm run build` in the `frontend` directory.

During codebase inspection, backend JavaScript syntax checks were also performed for app-owned backend files. The project currently does not include automated unit or integration tests; the backend `npm test` script remains a placeholder.

Current Verification Result

The frontend production build compiles successfully. Backend route/controller code has been reviewed structurally and supports the described workflows. Full end-to-end verification still requires a configured MongoDB instance and email credentials.

14. Limitations and Risks

- No automated test suite is currently implemented.
- CORS is currently permissive and should be restricted for production.
- OTP values are stored directly and should be hashed for production security.
- Some workflows depend on email service configuration, so registration and OTP flows require working SMTP/Gmail credentials.
- Uploads are stored on the local backend filesystem; cloud/object storage would be preferable for production.
- Payment, billing, doctor calendar synchronization, and notifications are not currently implemented.
- There is no deployment configuration in the current repository.

15. Future Scope

Testing

Add unit tests for controllers/services and integration tests for role-based access and appointment booking.

Security

Add rate limits, schema validation, stricter CORS, hashed OTP values, and audit logging.

Operations

Add deployment configuration, environment templates, logging, monitoring, and backup strategy.

User Experience

Add calendar view, notifications, report attachment uploads, appointment cancellation requests, and better search filters.

16. Conclusion

MeroSwasthya is a functional healthcare management platform with a practical role-based foundation. It supports the core workflows required by patients, doctors, hospitals, and administrators, including account verification, appointment booking, doctor-hospital relationships, medical reporting, and realtime communication.

The current implementation is suitable for demonstration, internal review, and continued development. Before production deployment, the recommended next priorities are automated testing, production security hardening, deployment configuration, and operational monitoring.